Architecture du backend

HBA Express — place de marche multivendeurs (Benin)

Monolithe modulaire .NET 8 | 25 modules | 1 API + 4 BFF

Ce document decrit la structure du backend telle qu'elle existe dans le depot, et non telle qu'elle avait ete prevue. Chaque chiffre a ete releve par lecture du code source : nombre de projets, de routes, de migrations, d'evenements. Les ecarts entre l'intention et la realite y sont signales comme tels.

100	25	5	614	38	81 800
projets .NET	modules metier	hotes web	routes HTTP	evenements	lignes C#

Sommaire

1. Principes d'architecture
2. Arborescence de la solution
3. Anatomie d'un module
4. Les 25 modules metier
5. Communication entre modules
6. Les hotes web : API et BFF

Architecture du backend

HBA Express — place de marche multivendeurs (Benin)

Monolithe modulaire .NET 8 | 25 modules | 1 API + 4 BFF

7. Blocs partages (BuildingBlocks)
8. Persistance et migrations
9. Traitements de fond
10. Tests et regles verifiees
11. Deploiement
12. Points de vigilance

1. Principes d'architecture

Le backend est un **monolithe modulaire** : un seul processus, une seule base de donnees, mais vingt-cinq modules cloisonnes qui ne se parlent qu'a travers des contrats publics. Le choix n'est pas ideologique. Une place de marche naissante n'a ni le trafic ni l'equipe que justifieraient des microservices ; elle a en revanche besoin de pouvoir en extraire un morceau le jour ou l'un d'eux devient un point chaud. Le cloisonnement prepare cette extraction sans en payer le prix aujourd'hui.

Quatre couches par module

Domain ne connait rien d'autre que lui-meme : entites, regles, evenements de domaine. Aucune reference a Entity Framework. **Application** orchestre — commandes et requetes MediatR, validation FluentValidation — sans savoir comment les donnees sont stockees. **Infrastructure** implemente : DbContext, configurations EF, migrations, appels sortants. **Contracts** est la seule porte ouverte aux autres modules.

Ces regles sont testees, pas seulement ecrites

Le projet `tests/Architecture.Tests` verifie par reflexion (NetArchTest) qu'aucune couche ne triche. Une dependance interdite fait echouer la compilation de la suite, pas une revue de code. C'est ce qui distingue une architecture d'une convention.

Regle verifiee	Ce qu'elle empeche
Domain ne depend pas d'EF Core	Que les regles metier soient contaminees par le stockage
Application ne depend ni d'EF Core ni de sa propre Infrastructure	Que l'orchestration court-circuite les abstractions
Application ne touche que les *Contracts* des autres modules	Le couplage direct entre modules — la fin du cloisonnement
Contracts n'expose aucun type interne	Qu'un detail d'implementation devienne un contrat de fait
Application ne depend pas des internes d'Identity	Que l'authentification fuite dans le reste du systeme

Socle technique

Element	Choix	Remarque
Plateforme	**.NET 8**	`TreatWarningsAsErrors` actif sur toute la solution
Mediation	MediatR 12.4	`ICommand` / `IQuery` maison, resultat `Result<T>`
Validation	FluentValidation 11.9	Branchee en *pipeline behavior*
Persistance	EF Core 8 + PostgreSQL	Un schema par module, une migration par module
Cache / temps reel	Redis + SignalR	SignalR adosse a Redis pour le multi-instance
Authentification	JWT + BCrypt + TOTP	`Otp.NET` pour la double authentification
Observabilite	Serilog + OpenTelemetry	Journal structure, metriques exportees
Images	SkiaSharp + service rembg	Detourage des photos produit
Notifications	FirebaseAdmin	Push mobile, acheteur et vendeur

Un reglage a connaitre avant tout le reste. `InvariantGlobalization` est active dans `Directory.Build.props`. Consequence non evidente : `string.Normalize(FormD)` devient une operation NEUTRE. Tout repliage d'accents doit donc se faire par table de correspondance explicite — c'est ce que fait `BeninGeography`, et c'est pourquoi il le fait.

2. Arborescence de la solution

100 projets, organises en quatre familles. La profondeur est reguliere : connaitre un module, c'est les connaitre tous.

```
marketPlace/
|
+-- Marketplace.sln                      100 projets
+-- Directory.Build.props                net8.0, nullable, warnings = erreurs
+-- Directory.Packages.props             versions centralisees
|
+-- src/
|   +-- BuildingBlocks/                  socle partage — 4 projets, 2 373 lignes
|   |   +-- ...Domain/                   Primitives, Results, Events, Geography
|   |   +-- ...Application/              Messaging, Behaviors, Pagination, Observability
|   |   +-- ...Infrastructure/           Outbox, Persistence, Caching, Files, Modularity
|   |   +-- ...IntegrationEvents/        contrat de l'evenement inter-modules
|   |
|   +-- Modules/                         25 modules x 4 projets — 58 401 lignes
|   |   +-- Catalog/
|   |   |   +-- Marketplace.Modules.Catalog.Domain/
|   |   |   +-- Marketplace.Modules.Catalog.Application/
|   |   |   +-- Marketplace.Modules.Catalog.Infrastructure/
|   |   |   +-- Marketplace.Modules.Catalog.Contracts/
|   |   +-- Ordering/  Payments/  Settlement/  ... (22 autres)
|   |
|   +-- Bootstrap/
|   |   +-- Marketplace.Api/             hote principal — 185 routes
|   |       +-- Endpoints/               25 fichiers, un par module
|   |       +-- Modules/                 enregistrement et composition
|   |       +-- Observability/           metriques, sante des dependances
|   |       +-- RealTime/                concentrateurs SignalR
|   |
|   +-- Bff/                             4 facades — 429 routes, 11 253 lignes
|       +-- Marketplace.Bff.Mobile/       91 routes  (application acheteur)
|       +-- Marketplace.Bff.Seller/      118 routes  (console et app vendeur)
|       +-- Marketplace.Bff.Admin/       171 routes  (console d'administration)
|       +-- Marketplace.Bff.Web/          49 routes  (site public)
|
+-- tests/                               29 projets — 4 929 lignes
+-- deploy/                              docker-compose, Ansible, Traefik
```

3. Anatomie d'un module

Exemple reel : le module **Ordering**. Les vingt-quatre autres suivent la meme forme, aux dossiers metier pres.

```
Ordering/
+-- Marketplace.Modules.Ordering.Domain/
|   +-- Orders/                    Order, OrderLine, OrderStatus, valeurs
|   +-- Orders/Events/             evenements de DOMAINE (internes au module)
|
+-- Marketplace.Modules.Ordering.Application/
|   +-- Abstractions/              interfaces de sortie (repositories, unit of work)
|   +-- Orders/Commands/           PlaceOrder, Cancel, Confirm, ...
|   +-- Orders/Queries/            GetOrder, ListOrders, ...
|   +-- Orders/EventHandlers/      reactions aux evenements des AUTRES modules
|
+-- Marketplace.Modules.Ordering.Infrastructure/
|   +-- Persistence/               OrderingDbContext (schema "ordering")
|   +-- Persistence/Configurations/ mapping EF, une classe par agregat
|   +-- Migrations/                9 migrations
|   +-- Public/                    implementation de IOrderingModuleApi
|
+-- Marketplace.Modules.Ordering.Contracts/
    +-- IOrderingModuleApi          lecture synchrone par les autres modules
    +-- OrderSummary, OrderLineSummary, ...   formes exposees
    +-- IntegrationEvents/          OrderPlaced, OrderConfirmed, OrderCancelled...
```

Le dossier `Public/` merite une mention : c'est la seule partie de l'Infrastructure qu'un autre module atteint, et il n'y accede qu'a travers l'interface declaree dans *Contracts*. L'implementation reste privee, l'interface est publique — la separation tient parce que les tests d'architecture la verifient.

Distinction a ne pas confondre

	Evenement de DOMAINE	Evenement d'INTEGRATION
Portee	Interne au module	Entre modules
Emission	Dans l'agregat	Publie via l'outbox
Transaction	Meme transaction	Apres validation, en differe
Ou il vit	`Domain/*/Events`	`Contracts/IntegrationEvents`

4. Les 25 modules metier

Chaque module possede son schema PostgreSQL et ses migrations propres. Le nombre de migrations est un indicateur utile : il dit ou le modele a le plus bouge, donc ou la comprehension du metier a le plus evolue.

Module	Schema	Migr.	Perimetre
Identity	`identity`	11	Comptes, roles, adresses, moyens de paiement enregistres
Catalog	`catalog`	3	Produits, marques, categories, medias
Offers	`offers`	4	Offres vendeur : prix, disponibilite, remises
Inventory	`inventory`	5	Stock, emplacements, reservations
Cart	`cart`	3	Paniers actifs
Ordering	`ordering`	9	Commandes, lignes, cycle de vie
Payments	`payments`	5	Paiements, passerelles PSP, webhooks
Settlement	`settlement`	12	Portefeuilles vendeurs, gains, lots de versement
Billing	`billing`	2	Commissions et factures
Pricing	`pricing`	3	Regles de prix et promotions
Tax	`tax`	2	Regles de TVA par juridiction
Shipping	`shipping`	3	Transporteurs, expeditions, zones de livraison
Returns	`returns`	4	Retours et remboursements
Disputes	`disputes`	3	Litiges acheteur / vendeur
Sellers	`sellers`	4	Boutiques, verification KYB
Reviews	`reviews`	3	Avis produit, moderation
Messaging	`messaging`	6	Conversations acheteur / vendeur
Notifications	`notifications`	4	Appareils, preferences, envois
Search	`search`	4	Index de documents
Recommendations	`recommendations`	2	Suggestions produit
Marketing	`marketing`	2	Campagnes et contenus
Loyalty	`loyalty`	2	Comptes de fidelite
Wishlist	`wishlist`	2	Listes d'envies
Analytics	`analytics`	2	Metriques agregees
Fraud	`fraud`	2	Evaluation du risque

Ce que revele la colonne des migrations. Settlement (12), Identity (11) et Ordering (9) concentrent a eux seuls un tiers des migrations de la solution. Ce sont les zones ou le modele a le plus tatonne — versements aux vendeurs, comptes, cycle de vie des commandes. Ce sont aussi celles a relire en premier avant toute reprise.

5. Communication entre modules

Deux voies, et une seule regle : jamais d'appel direct a l'interieur d'un autre module.

Voie 1 — lecture synchrone par *ModuleApi*

Seize interfaces `I...ModuleApi` exposent, dans *Contracts*, ce qu'un module accepte de reveler. L'appel est en processus : ni HTTP, ni serialisation, ni latence reseau. Le module appelant depend d'une interface, jamais d'une implementation.

```
ICartModuleApi        ICatalogModuleApi     ICommissionModuleApi   IFraudModuleApi
IIdentityModuleApi    IInventoryModuleApi   IMessagingModuleApi    IOfferModuleApi
IOrderingModuleApi    IPaymentsModuleApi    IPayoutModuleApi       IPricingModuleApi
IReviewsModuleApi     ISellerModuleApi      IShippingModuleApi     ITaxModuleApi
```

Voie 2 — evenements d'integration, via l'outbox

Trente-huit evenements. Ils ne sont pas publies directement : ils sont ecrits dans une table *outbox* **dans la meme transaction** que le changement metier, puis relayes par `OutboxProcessor`. C'est ce qui garantit qu'une commande ne peut pas etre enregistree sans que son evenement parte — ni l'inverse.

Commandes	**Catalogue**	**Stock / livraison**	**Comptes**
OrderPlaced	ProductCreated	StockReserved	UserRegistered
OrderConfirmed	ProductDeleted	StockDepleted	UserEmailConfirmed
OrderCancelled	ProductStatusChanged	ShipmentShipped	EmailVerificationRequested
OrderDelivered	BrandCreated	ShipmentDelivered	PasswordResetRequested
CartCheckedOut	CategoryCreated		SellerRegistered
	OfferCreated	**Apres-vente**	SellerActivated
Paiement	OfferDeleted	ReturnRefundApproved	SellerClosed
PaymentCaptured	OfferPriceChanged	ReturnRefunded	SellerReactivated
PaymentFailed	PromotionCreated	DisputeOpened	SellerDeleted
PaymentRefunded		DisputeResolved	
PayoutPaid			

38 evenements d'integration, regroupes par domaine d'origine.

La saga de commande

Le parcours d'achat traverse Cart, Ordering, Inventory, Payments, Shipping et Settlement. Il est couvert par `tests/Saga.IntegrationTests`, qui rejoue l'enchainement complet — c'est le seul endroit ou le systeme est verifie *en tant que systeme*, et non module par module.

6. Les hotes web : API et BFF

Cinq applications web se partagent le meme code metier. Les quatre BFF ne sont pas des passerelles HTTP : ils **referencent le projet** `Marketplace.Api.csproj` et appellent les modules en processus. Il n'y a donc aucun saut reseau entre un BFF et le metier.

Hote	Routes	Fichiers	Public servi
Marketplace.Api	185	25	Surface complete, un fichier d'endpoints par module. Porte aussi les webhooks PSP et les concentrateurs SignalR.
Bff.Admin	171	19	Console d'administration (Next.js)
Bff.Seller	118	20	Console vendeur (Next.js) et application vendeur (Flutter)
Bff.Mobile	91	14	Application acheteur (Flutter)
Bff.Web	49	10	Site public

A quoi sert un BFF, ici

A tailler la reponse pour un client precis, et a y placer les invariants qui n'appartiennent a aucun module. Le BFF Mobile, par exemple, agrege commande et expeditions en un seul appel — parce qu'un reseau mobile beninois rend chaque aller-retour couteux. Il refuse aussi une adresse de livraison incomplete avant meme d'ouvrir un paiement.

> **Une regle apprise a nos depens.** Un invariant metier place dans un seul BFF n'est pas un invariant : il ne protege que les clients qui passent par lui. Le controle d'adresse de livraison a d'abord vecu dans le BFF Mobile ; il a fallu le descendre dans `PlaceOrderCommandHandler` pour qu'il vaille reellement pour tout le monde. La regle qui en decoule : le BFF *anticipe* le refus pour donner un meilleur message, le domaine le *prononce*.

7. Blocs partages

2 373 lignes seulement, mais tout le reste s'appuie dessus. Un bloc partage qui grossit trop redevient un couplage : celui-ci est reste mince, delibrement.

Projet	Contenu	Ce qu'il apporte
Domain	Primitives, Results, Events, **Geography**	Entite et agregat de base, `Result<T>` et `Error`, `Money`, et le referentiel des 12 departements et 77 communes du Benin.
Application	Messaging, Behaviors, Pagination, Observability	`ICommand` / `IQuery`, validation et journalisation en pipeline, pagination uniforme.
Infrastructure	Outbox, Persistence, Caching, Files, Modularity, Serialization	Le processeur d'outbox, l'enregistrement des modules, le cache Redis, le stockage de fichiers.
IntegrationEvents	Contrat d'evenement	Le type de base commun a tous les evenements inter-modules.

Le referentiel geographique

`BuildingBlocks.Domain/Geography/BeninGeography.cs` merite d'etre signale, parce qu'il porte une decision structurante. Le Benin n'a pas de code postal operationnel et beaucoup de rues n'ont ni nom ni numero : une adresse s'y donne par **commune, quartier et point de repere**. La commune est donc choisie dans une liste fermee de 77 entrees, et c'est le **code** qui est stocke, jamais le libelle. On peut corriger une orthographe sans migrer une seule ligne.

8. Persistance et migrations

Une seule base PostgreSQL, **25 schemas**, un par module. Chaque module a son `DbContext`, ses configurations EF et ses migrations. Aucune cle etrangere ne traverse un schema : les references inter-modules sont des identifiants nus, resolus par appel de *ModuleApi*. C'est ce qui rendrait l'extraction d'un module possible sans demolir la base.

Total : **102 migrations** reparties sur 25 modules.

> **Les migrations s'appliquent au demarrage**, par `MigrateAsync()` — 25 appels, un par module — mais **seulement si** `RunMigrationsAtStartup` vaut `true`. En deploiement, ce drapeau est `true` pour l'API et `false` pour les quatre BFF : un seul processus migre, ce qui evite toute course entre hotes. C'est un bon reglage, et il merite d'etre connu — un BFF demarre avant l'API tournerait sur un schema non migre.
>
> Une migration en echec empeche donc l'API de demarrer. Toute migration ecrite a la main doit etre relue avec cela en tete, en particulier les changements de type de colonne, qui echouent si une seule valeur existante depasse la nouvelle contrainte.

Un point technique recurrent

Les migrations qui normalisent du texte n'utilisent pas l'extension `unaccent` — elle supposerait une extension installee sur chaque environnement. Elles emploient `translate()` avec une table de correspondance explicite, **identique** a celle du code C#. Les deux doivent rester alignees : une divergence entre le repliage SQL et le repliage C# produit des cles introuvables, sans aucune erreur.

9. Traitements de fond

Service	Role
`OutboxProcessor`	Relaie les evenements d'integration ecrits en base vers leurs abonnes. Coeur de la coherence entre modules.
`WithdrawalReconciliationService`	Rapproche les demandes de retrait vendeur de l'etat reel connu du prestataire.
`DiscountExpiryService`	Retire les remises arrivees a echeance.
`DependencyHealthPublisher`	Publie l'etat de sante des dependances (base, cache, PSP) sous forme de metriques.

`WithdrawalReconciliationService` est un precedent utile : il existe deja, pour les versements *sortants*, un service qui va redemander au prestataire ce qu'il est advenu d'une operation restee en suspens. Le meme besoin se pose en sens inverse, pour les encaissements Mobile Money qu'un acheteur valide apres que le prestataire a abandonne — cas connu, aujourd'hui non couvert.

10. Tests et regles verifiees

29 projets de tests, 4 929 lignes. Trois natures distinctes.

Nature	Projets	Ce qu'ils protegent
Tests unitaires	25 (un par module) + BuildingBlocks	Les regles metier de chaque agregat, isolees.
Tests d'integration	Catalog.IntegrationTests, Saga.IntegrationTests	Le parcours d'achat de bout en bout, et la composition complete du conteneur.
Tests d'architecture	Architecture.Tests	Les dependances entre couches et entre modules. Voir section 1.

11. Deploiement

Docker Compose derriere Traefik, orchestre par Ansible. Sept services applicatifs.

```
traefik            entree HTTPS, certificats ACME
  +-- api          Marketplace.Api
  +-- bff-mobile   application acheteur
  +-- bff-seller   console et application vendeur
  +-- bff-admin    console d'administration
  +-- bff-web      site public
  +-- console      console d'administration (Next.js)
  +-- seller-console  console vendeur (Next.js)
postgres           une base, 25 schemas          volume pgdata
redis              cache et backplane SignalR    volume redisdata
rembg              detourage des images produit  volume rembgmodels
```

Environnement	Domaine racine	Particularite
Production	`hba-express.org`	La console vendeur est servie sur `pro.hba-express.com` — une AUTRE racine, d'ou une variable dediee dans le compose.
Recette	`marketplace-staging.hba-marketplace.fr`	Meme composition, domaines suffixes.

12. Points de vigilance

Ce que ce document ne dirait pas si on se contentait de lire l'arborescence.

1. L'ordre de demarrage compte

Seule l'API migre (`RunMigrationsAtStartup=true`) ; les BFF ont le drapeau a `false`. Le reglage est bon, mais il cree une dependance implicite : un BFF qui demarre avant que l'API n'ait fini de migrer tourne sur un schema incomplet. Rien ne l'en empeche aujourd'hui.

2. InvariantGlobalization = true

`string.Normalize(FormD)` ne fait rien. Tout repliage d'accents passe par une table explicite, et cette table doit rester identique entre le C# et les migrations SQL. Une divergence ne leve aucune erreur — elle rend seulement des cles introuvables.

3. Un invariant dans un BFF n'est pas un invariant

Il ne protege que les clients qui empruntent ce BFF. Le domaine doit prononcer le refus ; le BFF ne fait que l'anticiper pour offrir un meilleur message.

4. Le stockage par code, jamais par libelle

Communes, statuts, methodes de paiement : le libelle peut changer, le code non. C'est ce qui permet de corriger une orthographe sans migration.

5. Pas de cle etrangere entre schemas

C'est ce qui garde l'extraction d'un module possible. La contrepartie est que l'integrite referentielle inter-modules n'est pas garantie par la base : elle repose sur le code et sur l'outbox.

6. Les paiements Mobile Money ont une longue traine

Un encaissement abandonne par le prestataire peut etre valide par l'acheteur plus tard. Sans reconciliation periodique, ce paiement reste invisible. Le precedent existe pour les versements sortants ; l'equivalent entrant reste a faire.

> **Sur la fidelite de ce document.** Les chiffres proviennent d'un comptage automatique du depot au 11 aout 2026 : projets, routes, migrations, evenements, lignes de code. Les perimetres de modules sont deduits de l'arborescence du domaine. Rien n'a ete complete de memoire — mais un comptage reste un comptage : il dit ce qui existe, pas si c'est utilise.